Artificial Intelligence (CS-203)

Objectives:

- **Problem Solving Agents**
- **Search Strategies**
  - **Some path Search algorithms**

Problem solving Agents

Problem-solving agents decide what to do by finding sequences of actions that lead to desirable states **(goals).**

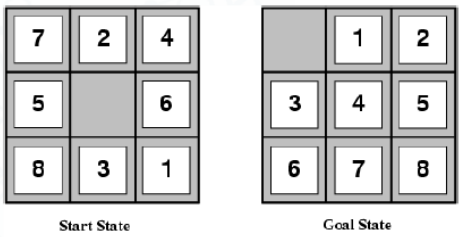
Problem Formulation

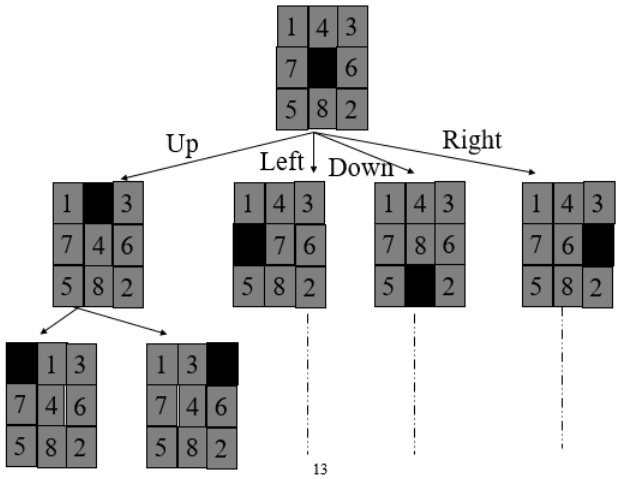
Problem formulation is the process of deciding what actions and states to consider, given a goal. The environment of the problem is represented by a state space. A path through the state space from the initial state to a goal state is a solution.

Example1:

In a sliding-tile puzzle, a number of tiles (sometimes called blocks or pieces) are arranged in a grid with one or more blank spaces so that some of the tiles can slide into the blank space. It has many variants one of which is **8-puzzle** which consists of a 3 x 3 grid with eight numbered tiles and one blank space, and the **15-puzzle** on a 4 x 4 grid.

- States
  - locations of tiles
- Actions
  - move blank left, right, up, down
- goal test
  - goal state (given)
- path cost
  - 1 per move





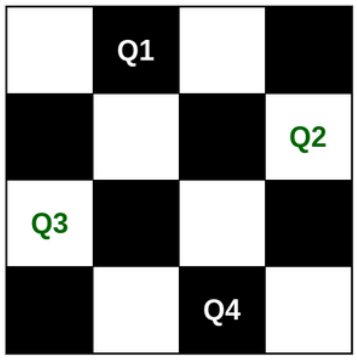
Example2:

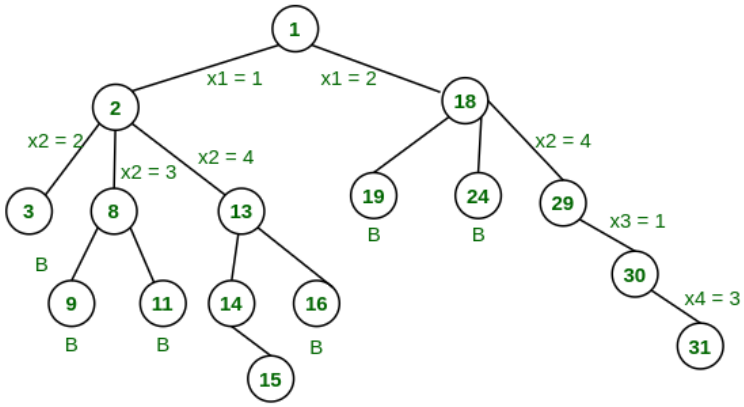
4 – Queens' problem is to place 4 – queens on a 4 x 4 chessboard in such a manner that no queens attack each other by being in the same row, column, or diagonal.

We will look for the solution for n=4 on a 4 x 4 chessboard. Here we have to place 4 queens say **Q1, Q2, Q3, Q4** on the 4 x 4 chessboard such that no 2 queens attack each other.

i. Let's suppose we're putting our first queen Q1 at position (1, 1) now for Q2 we can't put it in 1 row(because they will conflict).
ii. So for Q2 we will have to consider row 2. In row 2 we can place it in column 3 I.e at (2, 3) but then there will be no option for placing Q3 in row 3.
iii. So we **backtrack** one step and place Q2 at (2, 4) then we find the position for placing Q3 is (3, 2) but by this, no option will be left for placing Q4.
iv. Then we have to backtrack till 'Q1' and put it to (1, 2) instead of (1, 1) and then all other queens can be placed safely by moving Q2 to the position (2, 4), Q3 to (3, 1), and Q4 to (4, 3).

Hence we got our solution as **(2, 4, 1, 3)**, this is the one possible solution for the 4-Queen Problem. For another solution, we will have to backtrack to all possible partial solutions.

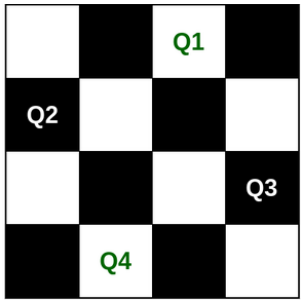




State Space Tree of 4 Queen Problem

The simple logic of generating the state space tree is to keep exploring all possible

The other possible solution for the 4 Queen problem is (3, 4, 1, 2)

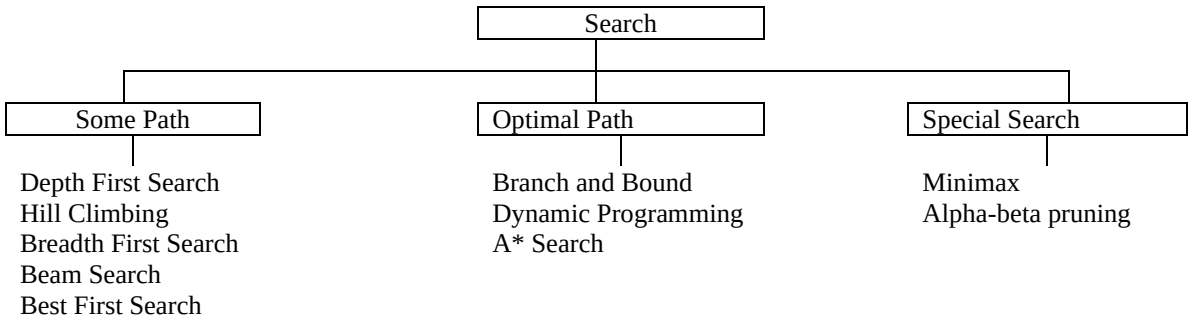


*****Searching graphs /trees is used as a Problem Solving technique. Any problem which can be converted into graph/tree its solution can be searched or found**

Types of Search Strategies

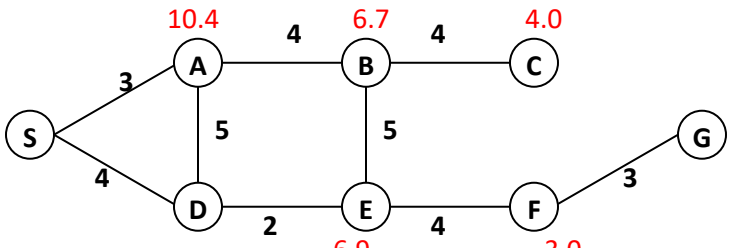
A search algorithm takes a search problem as input and returns a solution, or an indication of failure. We will consider algorithms that superimpose a search tree over the state-space graph, forming various paths from the initial state, trying to find a path that reaches a goal state. Each node in the search tree corresponds to a state in the state space and the edges in the search tree correspond to actions. The root of the tree corresponds to the initial state of the problem.

These notes will cover mainly three varieties of search techniques: (a) Search techniques which discover some path in a search space, mostly used to find paths from starting positions to goal positions, where the length of the discovered paths is not important, (b) Search techniques which discover some optimal path (i.e. the shortest or the cheapest path) in a search space, mostly used where the cost of traversing a path is of primary importance, and (c) Special search techniques, used in programs that play board games.



Finding Paths:

Finding paths can be exemplified by having a net of cities connected by highways, such as the net shown in the figure below:



The objective is to begin at city S, the starting point, and end at city G, the final goal. A sample path could be any one of the following: S-D-E-F-G, S-A-D-E-F-G-, S-A-B-E-F-G, S-D-A-B-E-F-G, and so on. Whereas the optimal path (or the shortest path) is only S-D-E-F-G. Finding a path involves two kinds of effort:

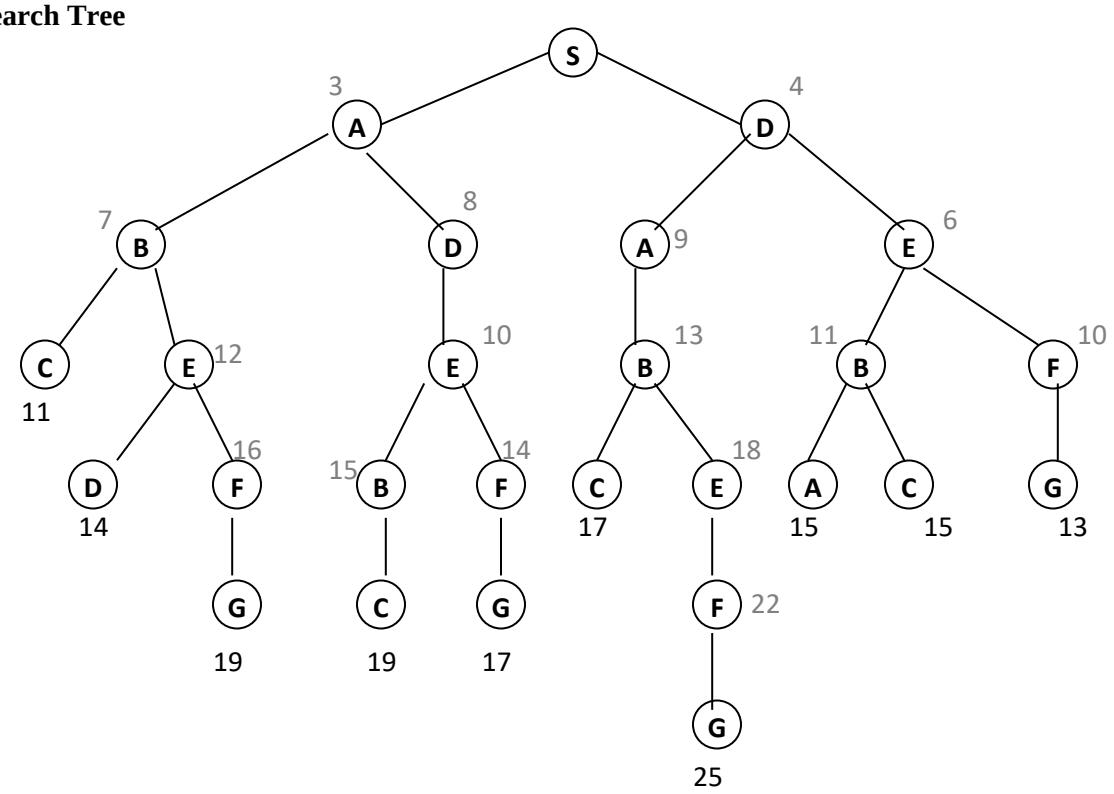
- Effort is spent in finding either some path or the shortest path.
- Effort is also spent in actually traversing the path or paths.

As Winston points out in his book, if it is necessary to go from Starting point to Goal point often, then it is worth a lot to find a really good path. On the other hand, if only one trip is required, and if the net is hard to force a way through, then it is proper to be content as soon as some path is found, even though better ones could be found with more work. Another thing, which is critical in

searching through a net, is to device a bookkeeping schemes that allows orderly exploration of all possible paths, and also takes care of cyclic paths such as S-A-D-A-D-S-A-D-..

With cyclic paths terminated, nets are equivalent to tree. The tree shown below is made from the above mentioned net, by following each possible path outward form the net's starting point "S" until it runs into a place already visited. The points in a net or tree are called *nodes* and the connections between *nodes* for nets and trees are called *links* and *branches* respectively. The costs at the end of the leaf nodes represent the overall cost in traversing that branch.

Search Tree



GENERIC SEARCH ALGORITHM

For all the search algorithms described below skeleton algorithm will be used, where a queue is used to perform the bookkeeping mentioned above:

To conduct a search:

1. Form a one-element queue consisting of the root node.
2. Until the queue is empty or the goal has been reached, determine if the first element in the queue is the goal node.
   - 2a If the first element is the goal node, ***stop*** *in case of <u>some path</u> search algorithms, and* ***continue*** *to search for the paths in case of <u>optimal path</u> search algorithms.*
   - 2b If the first element is not the goal node, than carry out the specific algorithm.
3. If the goal node has been found, announce success; otherwise announce failure.

Depth First Search

Depth first search takes a headlong dash to the bottom of the search tree along the leftmost branches. If the goal node is not found it backtracks to the previous nodes and tries the next branch. The specific algorithm followed by depth first search is (the overall algorithm is the *skeleton algorithm* mentioned above):

2b If the first element is not the goal node, remove the first element from the queue and add the first element's children, if any, to the **Start** of the queue.

This algorithm automatically allows headlong dash approach, with backtracking capability in case of reaching a non-goal leaf node. Tracing of depth first search algorithm on the example tree is shown below:

((S))
((SA) (SD))
((SAB) (SAD) (SD))
((SABC) (SABE) (SAD) (SD))
((SABE) (SAD) (SD))
((SABED) (SABEF) (SAD) (SD))
((SABEF) (SAD) (SD))
((SABEFG) (SAD) (SD))
Ans. : (SABEFG)

Breadth First Search

Breadth first search looks for the goal node among all nodes at a given level (i.e. siblings at the same level), before using children of those nodes to explore further. The specific algorithm for breadth first search resembles that of depth first search, differing only in the place where new elements are added to the queue, since that effect the exploration process:

2b If the first element is not the goal node, remove the first element from the queue, and add the first element's children, if any, to the ***End*** of the queue.

Tracing of Breadth First search algorithm is shown below:

((S))
((SA) (SD))
((SD) (SAB) (SAD))
((SAB) (SAD) (SDA) (SDE))
((SAD) (SDA) (SDE) (SABC) (SABE))
((SDA) (SDE) (SABC) (SABE) (SADE))
((SDE) (SABC) (SABE) (SADE) (SDAB))
((SABC) (SABE) (SADE) (SDAB) (SDEB) (SDEF))
((SABE) (SADE) (SDAB) (SDEB) (SDEF))
((SADE) (SDAB) (SDEB) (SDEF) (SABED) (SABEF))
((SDAB) (SDEB) (SDEF) (SABED) (SABEF) (SADEB) (SADEF))
((SDEB) (SDEF) (SABED) (SABEF) (SADEB) (SADEF) (SDABC) (SDABE))
((SDEF) (SABED) (SABEF) (SADEB) (SADEF) (SDABC) (SDABE) (SDEBA) (SDEBC))

((SABED) (SABEF) (SADEB) (SADEF) (SDABC) (SDABE) (SDEBA) (SDEBC) (SDEFG))
((SABEF) (SADEB) (SADEF) (SDABC) (SDABE) (SDEBA) (SDEBC) (SDEFG))
((SADEB) (SADEF) (SDABC) (SDABE) (SDEBA) (SDEBC) (SDEFG) (SABEFG))
((SADEF) (SDABC) (SDABE) (SDEBA) (SDEBC) (SDEFG) (SABEFG) (SADEBC))
((SDABC) (SDABE) (SDEBA) (SDEBC) (SDEFG) (SABEFG) (SADEBC) (SADEFG))
((SDABE) (SDEBA) (SDEBC) (SDEFG) (SABEFG) (SADEBC) (SADEFG))
((SDEBA) (SDEBC) (SDEFG) (SABEFG) (SADEBC) (SADEFG) (SDABEF))
((SDEBC) (SDEFG) (SABEFG) (SADEBC) (SADEFG) (SDABEF))
((SDEFG) (SABEFG) (SADEBC) (SADEFG) (SDABEF))
Ans. : (SDEFG)

Hill Climbing

Hill climbing is a heuristic or informed search. The aim of using heuristics in search process is to find the **better** solution or path from the start to the goal. A **Heuristic** is a rule of thumb for guessing which search nodes lead most quickly to the goal. Heuristics are imperfect short-cuts or estimates to improve performance.

Hill climbing is an improved version of **depth first search**, where the search takes a headlong dash to the bottom of the search tree, but instead of using all the leftmost branches, hill climbing makes the choice according to some heuristic measure of remaining distance. In other words it makes a choice in exploring the most promising path. The specific algorithm followed by hill climbing is (the overall algorithm is the *skeleton algorithm* mentioned above):

2b If the first element is not the goal node, remove the first element from the queue, sort the first element's children, if any, by the estimated remaining distance, and add them of the ***front*** of the queue.

Tracing of Hill Climbing search algorithm is shown below:

((S))
8.9 10.4
((SD) (SA))
6.9 10.4 10.4
((SDE) (SDA) (SA))
3.0 6.7 10.4 10.4
((SDEF) (SDEB) (SDA) (SA))
0.0 6.7 10.4 10.4
((SDEFG) (SDEB) (SDA)(SA))
Ans. : (SDEFG)

Beam Search

Beam search is like **breadth first search** because beam search progresses level by level. But, unlike breadth first search, beam search only expands (i.e. moves downward from) the best ***w*** nodes at each level. The following four figures graphically represent how beam search progress through the expansion process. Where ***w*** = 2, due to which certain nodes are ignored (with an **X**). The specific algorithm followed by beam search is:

2b If the first element is not the goal node, remove the first element from the queue, and if it fits the best ***w*** nodes criteria, then add its (i.e. the first element's) children, if any, to the back of the queue.

Tracing of Bream search algorithm is shown below:

((S))
8.9 10.4
((SD) (SA))
10.4 6.9
((SA) (SDE))
6.9 6.7
((SDE) (SAB))
6.7 3.0
((SAB) (SDEF))
3.0 4.0
((SDEF) (SABC))
4.0 0.0
((SABC) (SDEFG))
0.0
((SDEFG))

Ans. : (SDEFG)

Best First Search

Best first search is a modified form of **hill climbing,** which can be visualized as a team of co-operating hill climbing algorithms operating to seek out the highest point (or the optimal path) in the search space. In other words best first search tries to expand the search path from the best open node so far, no matter where it is in the partially developed tree.

The paths found by best first search are more likely to be shorter than those found with other methods, because best first search always moves forward from the node that seems closest to the goal node. However, this does not mean that it is certain to find the shortest path. The specific algorithm followed by best first search is:

2b If the first element is not the goal node, remove the first element from the queue, add the first element's children, if any, to the queue, and ***sort*** the entire queue by estimated remaining distance.

Tracing of Bream search algorithm is shown below:

((S))
8.9 10.4
((SD) (SA))
6.9 10.4 10.4
((SDE) (SDA) (SA))
3.0 6.7 10.4 10.4
((SDEF) (SDEB) (SDA) (SA))
0.0 6.7 10.4 10.4
((SDEFG) (SDEB) (SDA)(SA))

Ans. : (SDEFG)